Queue Variations — Beyond the Basics

Lecture 06

Data Structures and Algorithms

Part 1: The Circular Queue

Recall: The Wasted Space Problem

In Lecture 05, we saw a critical flaw in the linear array queue:

```
Step 1: enqueue 10, 20, 30, 40, 50    front=0, rear=4 (FULL)
[10 ][20 ][30 ][40 ][50 ]
  ^                   ^
  f                   r

Step 2: dequeue three times            front=3, rear=4
[ _ ][ _ ][ _ ][40 ][50 ]
                 ^    ^
                 f    r

Step 3: Try enqueue(60)?               rear == MAX-1 --> OVERFLOW!
[ _ ][ _ ][ _ ][40 ][50 ]     3 empty slots, but can't use them!
```

The queue reports "full" even though space exists — false overflow!

The shifting approach (O(n)) was too expensive. We need a better solution.

The Circular Queue Idea

What if the array wraps around?

Instead of stopping at the end of the array, we go back to the beginning!

```
Linear array:    [0] [1] [2] [3] [4]     --> hits a wall at index 4!

Circular array:  imagine bending the array into a ring:

             [0]
          /       \
       [4]       [1]
          \       /
       [3] --- [2]

After index 4, we wrap back to index 0!
```

Analogy: A carousel / merry-go-round — when you reach the last seat, the next one is the first seat again.

The Magic Formula: Modular Arithmetic

The key to wrapping around is the **modulo operator** `%`:

```
next_index = (current_index + 1) % MAX
```

Example with MAX = 5:

Current Index	`(index + 1) % 5`	Next Index
0	`(0+1) % 5 = 1`	1
1	`(1+1) % 5 = 2`	2
2	`(2+1) % 5 = 3`	3
3	`(3+1) % 5 = 4`	4
4	`(4+1) % 5 = 0`	**0 (wraps around!)**

This single formula replaces the "if at end, go to 0" logic with a clean, constant-time operation.

Circular Queue: Design

```c
#define MAX 5

struct CircularQueue {
    int arr[MAX];
    int front;
    int rear;
    int count;   // number of elements currently in the queue
};
```

Initialization: `front = 0`, `rear = -1`, `count = 0`

We use `count` to easily distinguish between **full** and **empty** states:

- **Empty:** `count == 0`
- **Full:** `count == MAX`

Some implementations use a "sacrifice one slot" approach instead, but `count` is cleaner.

Circular Queue: enqueue() Operation

Algorithm:

1. Check if queue is full (`count == MAX`) — if yes, overflow error
2. Advance `rear` using modular arithmetic: `rear = (rear + 1) % MAX`
3. Place element at `arr[rear]`
4. Increment `count`

```
void enqueue(struct CircularQueue* q, int value) {
    if (q->count == MAX) {
        printf("Queue Overflow!\n");
        return;
    }
    q->rear = (q->rear + 1) % MAX;
    q->arr[q->rear] = value;
    q->count++;
}
```

Time Complexity: O(1) — modular arithmetic is a constant-time operation.

Circular Queue: dequeue() Operation

Algorithm:

1. Check if queue is empty (`count == 0`) — if yes, underflow error
2. Read element at `arr[front]`
3. Advance `front` using modular arithmetic: `front = (front + 1) % MAX`
4. Decrement `count`

```c
int dequeue(struct CircularQueue* q) {
    if (q->count == 0) {
        printf("Queue Underflow!\n");
        return -1;
    }
    int value = q->arr[q->front];
    q->front = (q->front + 1) % MAX;
    q->count--;
    return value;
}
```

Time Complexity: O(1) — no shifting, no wasted space.

Circular Queue: Detailed Trace (MAX = 5)

```
Initial:  front=0, rear=-1, count=0
[ _ ][ _ ][ _ ][ _ ][ _ ]

enqueue(10): rear=(-1+1)%5=0, count=1
[10 ][ _ ][ _ ][ _ ][ _ ]
  ^
 f,r

enqueue(20): rear=1, count=2
[10 ][20 ][ _ ][ _ ][ _ ]
   ^    ^
   f    r

enqueue(30): rear=2, count=3
[10 ][20 ][30 ][ _ ][ _ ]
  ^         ^
  f         r

enqueue(40): rear=3, count=4
[10 ][20 ][30 ][40 ][ _ ]
   ^              ^
   f              r
```

So far, this looks exactly like a linear queue. The magic happens next...

Circular Queue: The Wrap-Around in Action

```
Continuing from: [10][20][30][40][ _ ]   front=0, rear=3, count=4

dequeue() → 10:   front=(0+1)%5=1, count=3
[ _ ][20 ][30 ][40 ][ _ ]
       ^         ^
       f         r

dequeue() → 20:   front=(1+1)%5=2, count=2
[ _ ][ _ ][30 ][40 ][ _ ]
             ^    ^
             f    r

enqueue(50): rear=(3+1)%5=4, count=3
[ _ ][ _ ][30 ][40 ][50 ]
             ^         ^
             f         r
```

Circular Queue: Wrapping Past the End!

```
Continuing from: [ _ ][ _ ][30][40][50]   front=2, rear=4, count=3

enqueue(60): rear=(4+1)%5=0, count=4      ← WRAPS AROUND TO INDEX 0!
[60 ][ _ ][30 ][40 ][50 ]
  ^         ^
  r         f

enqueue(70): rear=(0+1)%5=1, count=5      ← QUEUE IS NOW FULL!
[60 ][70 ][30 ][40 ][50 ]
       ^    ^
       r    f
```

The linear queue would have reported OVERFLOW at enqueue(50) itself!

But the circular queue successfully used the freed slots at indexes 0 and 1.

No wasted space. All operations O(1). Problem solved!

Circular Queue: peek() and Helper Functions

```c
int peek(struct CircularQueue* q) {
    if (q->count == 0) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q->arr[q->front];
}

int isEmpty(struct CircularQueue* q) {
    return q->count == 0;
}

int isFull(struct CircularQueue* q) {
    return q->count == MAX;
}

int size(struct CircularQueue* q) {
    return q->count;
}
```

All helper functions are **O(1)** — the `count` variable makes everything simple.

Alternative: Full Check Without Count

Some implementations avoid the `count` variable and instead check fullness using:

```
Full:   (rear + 1) % MAX == front
Empty:  front == rear (with a sentinel, or both at -1)
```

Trade-off: This approach sacrifices one array slot — a queue of size `MAX` can only hold `MAX - 1` elements.

```
Why? Because when all slots are occupied, the full condition
(rear + 1) % MAX == front looks the same as the empty condition!

Using count avoids this ambiguity and uses all MAX slots.
```

For exams and interviews, know **both** approaches!

Circular Queue: Complete Example

```c
int main() {
    struct CircularQueue q;
    q.front = 0;
    q.rear = -1;
    q.count = 0;

    enqueue(&q, 10);      // [10]
    enqueue(&q, 20);      // [10, 20]
    enqueue(&q, 30);      // [10, 20, 30]
    printf("%d\n", dequeue(&q));   // Output: 10
    printf("%d\n", dequeue(&q));   // Output: 20
    enqueue(&q, 40);      // [30, 40]
    enqueue(&q, 50);      // [30, 40, 50]
    enqueue(&q, 60);      // [30, 40, 50, 60]
    enqueue(&q, 70);      // [30, 40, 50, 60, 70] -- wraps around!
    printf("%d\n", peek(&q));      // Output: 30 (front element)
    printf("%d\n", size(&q));      // Output: 5 (full!)
}
```

Linear Queue vs Circular Queue

Feature	Linear Array Queue	Circular Queue
Wasted space	Yes (false overflow)	No — slots are reused
Wrap-around	No	Yes (via `% MAX`)
Full condition	`rear == MAX - 1`	`count == MAX`
Empty condition	`front > rear`	`count == 0`
Enqueue	O(1)	O(1)
Dequeue	O(1) but wastes space	O(1) with reuse
Space utilization	Poor after many dequeues	Optimal — 100% utilized

The circular queue is the standard array-based queue implementation — whenever someone says "queue using array" in practice, they almost always mean a circular queue.

Part 2: The Deque (Double-Ended Queue)

What is a Deque?

Deque (pronounced "**deck**", not "de-queue") stands for **Double-Ended Queue**.

It is a generalized queue that allows insertion and deletion at **both** ends.

```
                    insertFront()              insertRear()
                         |                          |
                         v                          v
              Front [10][20][30][40][50] Rear
                         ^                          ^
                         |                          |
                    deleteFront()              deleteRear()
```

Analogy: A deck of cards — you can draw from the top or the bottom!

Key insight: A Deque can act as **both** a Stack (use one end) and a Queue (use both ends in FIFO order). It is the most flexible linear data structure.

Deque Operations

Operation	Description
`insertFront(x)`	Add element `x` at the front
`insertRear(x)`	Add element `x` at the rear
`deleteFront()`	Remove and return the front element
`deleteRear()`	Remove and return the rear element
`getFront()`	View front element without removing
`getRear()`	View rear element without removing
`isEmpty()`	Check if deque is empty
`isFull()`	Check if deque is full (array-based)

Notice: A regular Queue only uses `insertRear` + `deleteFront`.

A Deque adds `insertFront` + `deleteRear` — operations from **both** ends.

Types of Deque

Input-Restricted Deque

```
Insertion only at ONE end; deletion at BOTH ends.

                                        insertRear()
                                             |
                                             v
              Front [10][20][30][40][50] Rear
                ^                          ^
                |                          |
           deleteFront()              deleteRear()
```

Use case: Situations where new items always enter from one side, but can be removed from either end.

Types of Deque (cont.)

Output-Restricted Deque

```
Insertion at BOTH ends; deletion only at ONE end.

          insertFront()              insertRear()
               |                          |
               v                          v
              Front [10][20][30][40][50] Rear
                ^
                |
            deleteFront()
```

Use case: Items can enter from either side, but always leave from one side.

Remember:

- **Input-restricted** = restricted *input* (insertion from one end only)
- **Output-restricted** = restricted *output* (deletion from one end only)

Deque: Implementation Strategies

A Deque can be implemented using:

1. Circular Array

- Similar to circular queue, but `front` and `rear` can move **in both directions**
- `insertFront()`: decrement `front` with wrap-around → `front = (front - 1 + MAX) % MAX`
- `insertRear()`: increment `rear` with wrap-around → `rear = (rear + 1) % MAX`
- All operations **O(1)**

2. Doubly Linked List (Natural Fit!)

- Each node has `prev` and `next` pointers
- Insert/delete at both head and tail in **O(1)**
- No size limitation, no wrapping logic

Doubly linked list is the most natural choice for a Deque since it supports bidirectional operations effortlessly.

Deque Using Doubly Linked List: Structure

```
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

struct Deque {
    struct Node* front;
    struct Node* rear;
    int count;
};
```

```
        NULL <-- [prev|10|next] <--> [prev|20|next] <--> [prev|30|next] --> NULL
                      ^                                          ^
                      |                                          |
                    front                                      rear
```

Both ends are immediately accessible — no traversal needed!

Deque: insertFront() and insertRear()

```
void insertFront(struct Deque* dq, int value) {
    struct Node* newNode = createNode(value);
    if (dq->front == NULL) {
        dq->front = dq->rear = newNode;
    } else {
        newNode->next = dq->front;
        dq->front->prev = newNode;
        dq->front = newNode;
    }
    dq->count++;
}

void insertRear(struct Deque* dq, int value) {
    struct Node* newNode = createNode(value);
    if (dq->rear == NULL) {
        dq->front = dq->rear = newNode;
    } else {
        newNode->prev = dq->rear;
        dq->rear->next = newNode;
        dq->rear = newNode;
    }
    dq->count++;
}
```

Deque: deleteFront() and deleteRear()

```c
int deleteFront(struct Deque* dq) {
    if (dq->front == NULL) { printf("Underflow!\n"); return -1; }
    struct Node* temp = dq->front;
    int value = temp->data;
    dq->front = dq->front->next;
    if (dq->front == NULL)
        dq->rear = NULL;          // Deque became empty
    else
        dq->front->prev = NULL;
    free(temp);
    dq->count--;
    return value;
}

int deleteRear(struct Deque* dq) {
    if (dq->rear == NULL) { printf("Underflow!\n"); return -1; }
    struct Node* temp = dq->rear;
    int value = temp->data;
    dq->rear = dq->rear->prev;
    if (dq->rear == NULL)
        dq->front = NULL;          // Deque became empty
    else
        dq->rear->next = NULL;
    free(temp);
    dq->count--;
    return value;
}
```

Deque: Visualization

```
Initial: empty deque (front = NULL, rear = NULL)

insertRear(10):
   front --> [10] <-- rear

insertRear(20):
   front --> [10] <--> [20] <-- rear

insertFront(5):
   front --> [5] <--> [10] <--> [20] <-- rear

deleteRear() returns 20:
   front --> [5] <--> [10] <-- rear

deleteFront() returns 5:
   front --> [10] <-- rear

insertFront(1):
   front --> [1] <--> [10] <-- rear
```

All operations O(1) — just pointer rewiring, no traversal!

Where Does the Deque Fit?

A Deque is a **superset** of both Stack and Queue:

```
+---------------------------+
|          DEQUE            |
|  (insert/delete both ends) |
|                           |
|   +-------+  +-------+    |
|   | STACK |  | QUEUE |    |
|   | (one  |  | (FIFO |    |
|   |  end) |  |  order)|   |
|   +-------+  +-------+    |
+---------------------------+
```

- **Use only one end** → Deque behaves as a **Stack**
- **Insert at rear, delete from front** → Deque behaves as a **Queue**
- **Use both ends freely** → Full **Deque** capability

Part 3: The Priority Queue

What is a Priority Queue?

A Priority Queue is **not** a standard FIFO queue — elements are served based on **priority**, not arrival order.

Analogy: Hospital Emergency Room

- A patient with a heart attack is treated before someone with a sprained ankle, **regardless of who arrived first**

Another analogy: Airline boarding

- Business class boards before economy, even if economy passengers checked in earlier

Every element has:

- A **value** (the data)
- A **priority** (a number — lower or higher means more urgent depending on type)

Priority Queue: How It Works

```
Regular Queue (FIFO):     Priority Queue:

Arrive in order:          Arrive in order:
  P1, P2, P3, P4           P1(pri=3), P2(pri=1), P3(pri=4), P4(pri=2)

Served in order:          Served in priority order:
  P1, P2, P3, P4           P2(pri=1), P4(pri=2), P1(pri=3), P3(pri=4)
                            ↑ lowest number = highest priority (min-priority)
```

Key difference: In a regular queue, the **oldest** element is served first. In a priority queue, the element with the **highest priority** is served first, regardless of when it arrived.

Types of Priority Queue

Ascending Priority Queue (Min-Priority)

- **Smallest** priority number = **highest** priority → served first
- Example: Priority 1 is served before Priority 5
- Used in: Dijkstra's shortest path, Huffman coding

Descending Priority Queue (Max-Priority)

- **Largest** priority number = **highest** priority → served first
- Example: Priority 10 is served before Priority 3
- Used in: Job scheduling (higher priority jobs first)

```
Min-Priority Queue:  dequeue always returns the element with SMALLEST priority
Max-Priority Queue:  dequeue always returns the element with LARGEST priority
```

Priority Queue: Implementation Approaches

There are several ways to implement a Priority Queue, each with different trade-offs:

Implementation	Insert	Delete (highest priority)	Peek
Unsorted Array	O(1)	O(n) — must scan for min/max	O(n)
Sorted Array	O(n) — must insert in order	O(1) — min/max is at one end	O(1)
Unsorted Linked List	O(1)	O(n) — must scan	O(n)
Sorted Linked List	O(n) — must find correct position	O(1) — head has highest priority	O(1)
Binary Heap ⭐	O(log n)	O(log n)	O(1)

The Binary Heap is the best approach — we will study it in depth in the Trees unit. For

Priority Queue: Using an Unsorted Array

Idea: Just append elements at the end. When deleting, scan the entire array to find the highest-priority element.

```
#define MAX 100

struct PriorityQueue {
    int data[MAX];
    int priority[MAX];   // lower value = higher priority
    int size;
};

void insert(struct PriorityQueue* pq, int value, int pri) {
    if (pq->size == MAX) { printf("Overflow!\n"); return; }
    pq->data[pq->size] = value;
    pq->priority[pq->size] = pri;
    pq->size++;
}
// Insert is O(1) — just append!
```

Priority Queue: Unsorted Array — Delete

```c
int deleteHighestPriority(struct PriorityQueue* pq) {
    if (pq->size == 0) { printf("Underflow!\n"); return -1; }

    // Find element with the smallest priority value (highest priority)
    int minIdx = 0;
    for (int i = 1; i < pq->size; i++) {
        if (pq->priority[i] < pq->priority[minIdx]) {
            minIdx = i;
        }
    }

    int value = pq->data[minIdx];

    // Shift last element into the gap (order doesn't matter)
    pq->data[minIdx] = pq->data[pq->size - 1];
    pq->priority[minIdx] = pq->priority[pq->size - 1];
    pq->size--;

    return value;
}
// Delete is O(n) — must scan the entire array!
```

Priority Queue: Using a Sorted Array

Idea: Keep the array sorted by priority. Insert requires finding the correct position, but delete is instant.

```
Insert(value=X, priority=3) into sorted array:

Before:  [(A,1), (B,2), (D,4), (E,5)]     ← sorted by priority

Find position for priority 3:
         [(A,1), (B,2), ___, (D,4), (E,5)]

Shift right and insert:
         [(A,1), (B,2), (X,3), (D,4), (E,5)]

Delete highest priority:
         Just remove index 0!  → returns A (priority 1)
         [(B,2), (X,3), (D,4), (E,5)]
```

Insert: O(n) (shifting), **Delete: O(1)** (just remove from one end).

Priority Queue: Trade-off Summary

```
Unsorted approaches:
  ✓ Insert is fast (O(1))
  ✗ Delete is slow (O(n)) — need to search for highest priority

Sorted approaches:
  ✗ Insert is slow (O(n)) — need to maintain sorted order
  ✓ Delete is fast (O(1)) — highest priority is at one end

Binary Heap (best of both worlds):
  ✓ Insert is O(log n)
  ✓ Delete is O(log n)
  → Covered in the Trees unit!
```

When to use which?

- Few deletions, many inserts → Unsorted approach
- Few inserts, many deletions → Sorted approach
- Balanced workload → Binary Heap (the standard choice)

Priority Queue: Applications

Priority Queues are everywhere in computer science:

1. CPU Scheduling

- Processes with higher priority get CPU time first
- The OS maintains a priority queue of ready processes

2. Dijkstra's Shortest Path Algorithm

- A min-priority queue selects the unvisited node with the smallest distance
- Efficiently finds shortest paths in weighted graphs

3. Huffman Coding (Data Compression)

- Builds an optimal prefix code by repeatedly extracting the two lowest-frequency nodes

4. Event-Driven Simulations

- Events are processed in chronological order using a priority queue sorted by time

Part 4: The Queue Family — Putting It All Together

The Queue Family Tree

```
                    Queue (FIFO)
                   /     |      \
                  /      |       \
          Linear     Circular     Deque
          Queue      Queue        (Double-Ended)
          (basic)    (no wasted    /          \
                      space)   Input-       Output-
                               Restricted   Restricted

                               Priority Queue
                               (not FIFO — by priority)
```

Each variation addresses a specific limitation or adds new capability:

- **Circular Queue** → fixes wasted space
- **Deque** → adds flexibility (both ends)
- **Priority Queue** → adds ordering by importance

When to Use Which Queue?

Scenario	Best Queue Type
Simple task processing in order	**Linear Queue** (or Circular)
Fixed-size buffer with reuse	**Circular Queue**
Need to add/remove from both ends	**Deque**
Sliding window problems	**Deque**
Process items by importance	**Priority Queue**
CPU scheduling (time slicing)	**Circular Queue**
Shortest path in weighted graphs	**Priority Queue**
Undo + Redo operations	**Deque** (as a stack from both ends)

Part 5: Tracing Through Examples

Trace Exercise 1: Circular Queue (MAX = 4)

Trace the operations: `enqueue(A), enqueue(B), enqueue(C), dequeue(), dequeue(), enqueue(D), enqueue(E), enqueue(F)`

```
enqueue(A): rear=(-1+1)%4=0, count=1     [ A ][ _ ][ _ ][ _ ]   f=0, r=0
enqueue(B): rear=(0+1)%4=1,  count=2     [ A ][ B ][ _ ][ _ ]   f=0, r=1
enqueue(C): rear=(1+1)%4=2,  count=3     [ A ][ B ][ C ][ _ ]   f=0, r=2
dequeue()→A: front=(0+1)%4=1, count=2    [ _ ][ B ][ C ][ _ ]   f=1, r=2
dequeue()→B: front=(1+1)%4=2, count=1    [ _ ][ _ ][ C ][ _ ]   f=2, r=2
enqueue(D): rear=(2+1)%4=3,  count=2     [ _ ][ _ ][ C ][ D ]   f=2, r=3
enqueue(E): rear=(3+1)%4=0,  count=3     [ E ][ _ ][ C ][ D ]   f=2, r=0  ← WRAP!
enqueue(F): rear=(0+1)%4=1,  count=4     [ E ][ F ][ C ][ D ]   f=2, r=1  ← FULL!
```

Trace Exercise 1: Visual Representation

The final state in circular form (MAX = 4):

```
            [0] = E
          /         \
    [3] = D         [1] = F
          \         /
            [2] = C  ← front

    front = 2,  rear = 1,  count = 4  (FULL)
```

If we now dequeue all:

- `dequeue()` → C (front moves to 3)
- `dequeue()` → D (front moves to 0)
- `dequeue()` → E (front moves to 1)
- `dequeue()` → F (front moves to 2, count = 0 → EMPTY)

Elements come out in FIFO order: C, D, E, F — matching their enqueue order.

Trace Exercise 2: Deque Operations

Trace: `insertRear(10), insertRear(20), insertFront(5), insertRear(30), deleteFront(), deleteRear()`

```
insertRear(10):
   front --> [10] <-- rear

insertRear(20):
   front --> [10] <--> [20] <-- rear

insertFront(5):
   front --> [5] <--> [10] <--> [20] <-- rear

insertRear(30):
   front --> [5] <--> [10] <--> [20] <--> [30] <-- rear

deleteFront() returns 5:
   front --> [10] <--> [20] <--> [30] <-- rear

deleteRear() returns 30:
   front --> [10] <--> [20] <-- rear
```

Trace Exercise 3: Priority Queue (Min-Priority)

Insert elements with priorities, then delete highest priority (smallest number):

```
insert(Task_A, priority=3)    →  [(A,3)]
insert(Task_B, priority=1)    →  [(A,3), (B,1)]
insert(Task_C, priority=4)    →  [(A,3), (B,1), (C,4)]
insert(Task_D, priority=2)    →  [(A,3), (B,1), (C,4), (D,2)]

deleteHighestPriority()       →  returns Task_B (priority 1)
                                  [(A,3), (D,2), (C,4)]

deleteHighestPriority()       →  returns Task_D (priority 2)
                                  [(A,3), (C,4)]

deleteHighestPriority()       →  returns Task_A (priority 3)
                                  [(C,4)]
```

Elements are served by priority, not by arrival order!

Key Takeaways

What We Learned Today

1. Circular Queue

- Solves the wasted-space problem of linear array queues
- Uses modular arithmetic: `(index + 1) % MAX` for wrapping
- All operations remain **O(1)**
- The standard array-based queue in practice

2. Deque (Double-Ended Queue)

- Insert and delete at **both** ends
- Can simulate both a Stack and a Queue
- Two types: Input-Restricted and Output-Restricted
- Best implemented with a **doubly linked list** — all operations O(1)

What We Learned Today (cont.)

3. Priority Queue

- Elements served by **priority**, not arrival order
- Two types: Min-Priority (smallest first) and Max-Priority (largest first)
- Simple implementations: unsorted array (O(1) insert, O(n) delete) or sorted array (O(n) insert, O(1) delete)
- Best implementation: **Binary Heap** — O(log n) for both (covered in Trees unit)

4. The Queue Family

- Linear Queue → Circular Queue → Deque → Priority Queue
- Each solves a specific problem or adds new capabilities
- Choosing the right variant depends on your use case

Complexity Summary

Data Structure	Insert	Delete	Peek	Space
Linear Queue (Array)	O(1)	O(1)*	O(1)	O(n)
Circular Queue	O(1)	O(1)	O(1)	O(n)
Deque (Doubly LL)	O(1)	O(1)	O(1)	O(n)
Priority Queue (Unsorted)	O(1)	O(n)	O(n)	O(n)
Priority Queue (Sorted)	O(n)	O(1)	O(1)	O(n)
Priority Queue (Heap)	O(log n)	O(log n)	O(1)	O(n)

*Linear array dequeue wastes space; circular queue is the proper fix.

Review Questions

1. What is the "wasted space" problem, and how does the circular queue solve it?
2. Write the formula used for wrapping around in a circular queue. Why does it work?
3. What is the difference between an Input-Restricted Deque and an Output-Restricted Deque?
4. In a Priority Queue, what is the difference between ascending (min-priority) and descending (max-priority)?
5. Compare the time complexities of inserting into a Priority Queue using (a) unsorted array, (b) sorted array, and (c) binary heap.
6. Can a Deque behave as a Stack? As a Queue? Explain how.
7. Given a circular queue of MAX = 4 with `front = 3`, `rear = 1`, and `count = 3`, what are the elements' indices?